CM0081 Formal Languages and Automata
§ 9.2 An Undecidable Problem That Is Recursively Enumerable

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2024-1

(Last modification: 12th January 2026)

Preliminaries

Textbook

John E. Hopcroft, Rajeev Motwani and Jefferey D. Ullman (2007). Introduction to Automata Theory, Languages, and Computation.

Conventions

- The number and page numbers assigned to chapters, examples, exercises, figures, quotes, sections and theorems on these slides correspond to the numbers assigned in the textbook.
- The natural numbers include the zero, that is, $\mathbb{N} = \{0, 1, 2, \dots\}$.
- The power set of a set A, that is, the set of its subsets, is denoted by $\mathcal{P}A$.

Theorems About Recursive Languages

Theorem 9.3

If L is a recursive language, then $\overline{L}$ is also a recursive language.

Proof

Whiteboard.

Theorems About Recursive Languages

Theorem 9.3
If L is a recursive language, then $\overline{L}$ is also a recursive language.

Proof
Whiteboard.

Theorem 9.4
If both L and $\overline{L}$ are recursively enumerable languages, then L is recursive (and $\overline{L}$ is recursive as well by Theorem 9.3).

Proof
Whiteboard.

Theorems About Recursive Languages

Possible relations between a language L and its complement $\overline{L}$

(i) Both L and $\overline{L}$ are recursive.

Theorems About Recursive Languages

Possible relations between a language L and its complement $\overline{L}$

(i) Both L and $\overline{L}$ are recursive.

(ii) Neither L nor $\overline{L}$ are recursively enumerable.

Theorems About Recursive Languages

Possible relations between a language L and its complement $\overline{L}$

(i) Both L and $\overline{L}$ are recursive.

(ii) Neither L nor $\overline{L}$ are recursively enumerable.

(iii) L is recursively enumerable but not recursive and $\overline{L}$ is not recursively enumerable.

Theorems About Recursive Languages

Possible relations between a language L and its complement $\overline{L}$

(i) Both L and $\overline{L}$ are recursive.

(ii) Neither L nor $\overline{L}$ are recursively enumerable.

(iii) L is recursively enumerable but not recursive and $\overline{L}$ is not recursively enumerable.

(iv) $\overline{L}$ is recursively enumerable but not recursive and L is not recursively enumerable.

Theorems About Recursive Languages

Exercise 9.2.5

Let L be recursively enumerable and let $\overline{L}$ be non recursively enumerable. Consider the language

$$L' = \{\, 0w \mid w \text{ is in } L \,\} \cup \{\, 1w \mid w \text{ is not in } L \,\}.$$

Can you say for certain whether L' is recursive, recursively enumerable, or non recursively enumerable? Justify your answer.

Solution (from Hopcroft, Motwani and Ullman (2007))

Suppose L' were recursively enumerable. Then we could design a Turing machine M for $\overline{L}$ as follows. Given input w, M changes its input to $1w$ and simulates the hypothetical Turing machine for L'. If that Turing machine accepts, then w is in $\overline{L}$, so M should accept. If the Turing machine for L' never accepts, then neither does M. Thus, M would accept exactly $\overline{L}$, which contradicts the fact that $\overline{L}$ is not recursively enumerable. We conclude that L' is not recursively enumerable.

The Universal Language

Conventions

1. (M, w): Represents the Turing machine M with input w.
2. w is a string of 0's and 1's.

Codification of a Turing machine with an input

Let w_i be the codification of a Turing machine M. The codification of (M, w) is defined by

$$\overrightarrow{(M, w)} := w_i 111 w.$$

The Universal Language

Definition

Let $\Sigma = \{0, 1\}$. The **universal language**, denoted $\mathrm{L_u}$, is the set of binary strings that encode a pair (M, w) such that $w \in \mathrm{L}(M)$, that is,

$$\mathrm{L_u} := \left\{ \overrightarrow{(M, w)} \in \Sigma^* \;\middle|\; w \in \mathrm{L}(M) \right\}.$$

The Universal Language

Theorem

The language L_u is recursively enumerable.

Idea of the proof

There exists a Turing machine U such that $L_u = L(U)$. The machine U is called a **universal Turing machine**.

The Universal Language

Theorem 9.6

The language $\mathrm{L_u}$ is recursively enumerable but not recursive.

Proof of $\mathrm{L_u}$ is not recursive (by contradiction (proof of negation))

Suppose $\mathrm{L_u}$ is recursive

$\Rightarrow$ $\overline{\mathrm{L_u}}$ is recursive

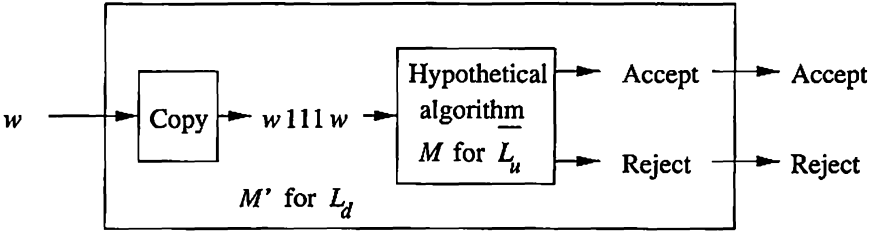
$\Rightarrow$ $\mathrm{L_d}$ is recursive (see next slide)

$\Rightarrow$ Contradiction because $\mathrm{L_d}$ is non recursively enumerable ■

The Universal Language

From the recursiveness of $\overline{\mathrm{L_u}}$ to the recursiveness of $\mathrm{L_d}$

Given a terminating Turing machine for accepting $\overline{\mathrm{L_u}}$ we could use this machine for building a terminating Turing machine for accepting $\mathrm{L_d}$ (Fig. 9.6).



Since $\mathrm{L_d}$ is not recursive (because it is not recursive enumerable) then $\overline{\mathrm{L_u}}$ is not recursive.

Code for a Universal Turing Machine

Code for U

Since U is a Turing machine exists i (1654 digits) such that $\mathrm{U} = M_i$ given by (using a different codification) (Penrose 1991, pp. 56-57):

724485533533931757719839503961571123795236067255655963110814479660650
505940424109031048361363235936564444345838222688327876762655614469281
411771501784255170755408565768975334635694247848859704693472573998858
228382779529468346052106116983594593879188554632644092552550582055598
945189071653741489603309675302043155362503498452983232065158304766414
213070881932971723415105698026273468642992183817215733348282307345371
342147505974034518437235959309064002432107734217885149276079759763441
512307958639635449226915947965461471134570014504816733756217257346452
273105448298078496512698878896456976090663420447798902191443793283001
949357096392170390483327088259620130177372720271862591991442827543742

(continued on next slide)

Code for a Universal Turing Machine

2351355675134084222299889374410534305471044368695876405178128019437530
8138706399427728231564252892375145654438990527807932411448261423572861
9311833261065612275553181020751108533763380603108236167504563585216
4214869542347187426437544428790062485827091240422076538754264454133451
7485662915742999095026230097337381377241621727477236102067868540028935
6608569682262014198248621698902609130940298570600174300670086896759
0344734174127874255812015493663938996905817738591654055356704092821332
2216314109787108145997866959970450968184190629944365601514549048809220
8448003482249207730403043188429899393135266882349662101947161910701
4619685231928474820344958977095535611070275817487333272966789987984732
8409819076485127263100174016678736347760585724503696443489799203448999
7455662402937487668839751404451665707750060513883991668814072545544
6652220507242623923792115253181625125363050931728631422004064571305275
8023076651833519956891397481375049264296050100136519801869456394 98

Turing's Universal Turing Machine

- Based on M-functions (subroutines with parameters) (Sicard 1997; Copeland 2004).
- The machine is composed by 12 symbols and 4.000 instructions, approximately (Sicard Ramírez 1998).

Small Universal Turing Machines

Notation

Let $\mathrm{UTM}(m, n)$ be the class of universal Turing machines with m states and n symbols.

Small Universal Turing Machines

Notation

Let $\mathrm{UTM}(m, n)$ be the class of universal Turing machines with m states and n symbols.

Theorem

If $\mathrm{UTM}(m, n) \neq \emptyset$ then (Shannon 1956):

(i) $\mathrm{UTM}(2, n') \neq \emptyset$, where n' is at most $4mn + n$ and

(ii) $\mathrm{UTM}(m', 2) \neq \emptyset$, where $m' = (2^l - 1)m$ and l is the smaller integer such that $m \leq 2^l$.

Small Universal Turing Machines

Theorem

There exists universal Turing machines in the following classes (Rogozhin 1996; Neary and Woods 2012):

$\mathrm{UTM}(m, n)$	Author(s)
$(24, 2)$	Rogozhin (1996)
$(19, 2)$	Baiocchi (2001)
$(18, 2)$	Neary and Woods (2007)
$(15, 2)$	Neary and Woods (2009)

(continued on next slide)

Small Universal Turing Machines

Theorem (continuation)

$\mathrm{UTM}(m, n)$	Author(s)
$(10, 3)$	Rogozhin (1996)
$(9, 3)$	Neary and Woods (2009)
$(7, 4)$	Rogozhin (1996)
$(6, 4)$	Neary and Woods (2009)
$(5, 5)$	Rogozhin (1996)
$(4, 6)$	Rogozhin (1996)
$(3, 10)$	Rogozhin (1996)
$(2, 18)$	Rogozhin (1996)

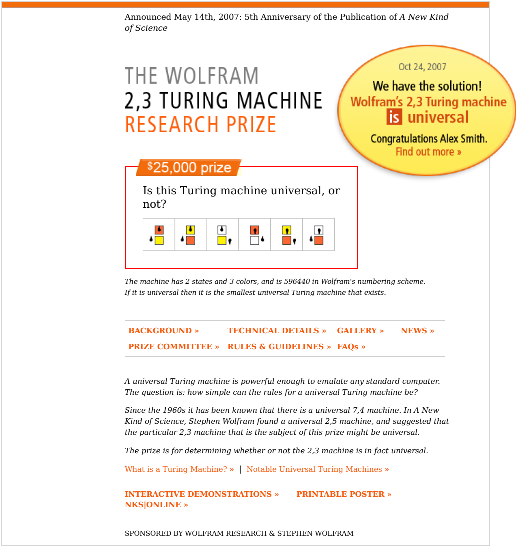
Small Universal Turing Machines

Theorem

The following classes are empty (Rogozhin 1996; Neary and Woods 2012):

$\mathrm{UTM}(m, n)$	Author(s)
$(m, 1)$	trivial
$(3, 2)$	Rogozhin (1996)
$(2, 3)$	Rogozhin (1996)
$(2, 2)$	Rogozhin (1996)
$(1, n)$	Herman (1968)

Wolfram Turing Machine


Announced May 14th, 2007: 5th Anniversary of the Publication of *A New Kind of Science*

THE WOLFRAM
2,3 TURING MACHINE
RESEARCH PRIZE

Oct 24, 2007
We have the solution!
Wolfram's 2,3 Turing machine is universal
Congratulations Alex Smith.
Find out more »

$25,000 prize

Is this Turing machine universal, or not?

The machine has 2 states and 3 colors, and is 596440 in Wolfram's numbering scheme. If it is universal then it is the smallest universal Turing machine that exists.

BACKGROUND » TECHNICAL DETAILS » GALLERY » NEWS »
PRIZE COMMITTEE » RULES & GUIDELINES » FAQs »

A universal Turing machine is powerful enough to emulate any standard computer. The question is: how simple can the rules for a universal Turing machine be?

Since the 1960s it has been known that there is a universal 7,4 machine. In A New Kind of Science, Stephen Wolfram found a universal 2,5 machine, and suggested that the particular 2,3 machine that is the subject of this prize might be universal.

The prize is for determining whether or not the 2,3 machine is in fact universal.

What is a Turing Machine? » | Notable Universal Turing Machines »

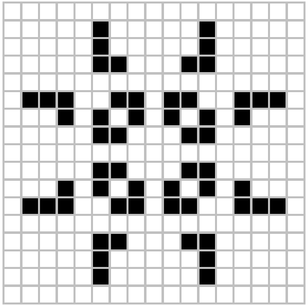
INTERACTIVE DEMONSTRATIONS » PRINTABLE POSTER »
NKS|ONLINE »

SPONSORED BY WOLFRAM RESEARCH & STEPHEN WOLFRAM

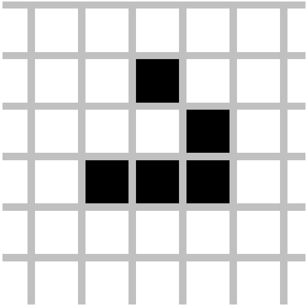
Conway's Game of Life

Examples

(From Wikipedia)



Pulsar (Oscillator)
Author: Jokey Smurf



Glider (Spaceship)
Author: Rodrigo Silveira Camargo

Conway's Game of Life

Rules

(i) Any live cell with fewer than two live neighbours dies, as if caused by under-population.

Conway's Game of Life

Rules

(i) Any live cell with fewer than two live neighbours dies, as if caused by under-population.

(ii) Any live cell with two or three live neighbours lives on to the next generation.

Conway's Game of Life

Rules

(i) Any live cell with fewer than two live neighbours dies, as if caused by under-population.

(ii) Any live cell with two or three live neighbours lives on to the next generation.

(iii) Any live cell with more than three live neighbours dies, as if by overcrowding.

Conway's Game of Life

Rules

(i) Any live cell with fewer than two live neighbours dies, as if caused by under-population.

(ii) Any live cell with two or three live neighbours lives on to the next generation.

(iii) Any live cell with more than three live neighbours dies, as if by overcrowding.

(iv) Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.

Conway's Game of Life

Rules

(i) Any live cell with fewer than two live neighbours dies, as if caused by under-population.

(ii) Any live cell with two or three live neighbours lives on to the next generation.

(iii) Any live cell with more than three live neighbours dies, as if by overcrowding.

(iv) Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.

Theorem

It is possible codified a universal Turing machine in Conway's Game of Life (Rendell 2011).

The Halting Problem

The language of the halting problem

Let $\Sigma = \{0, 1\}$. The original Turing machine accepted by halting, no by final state.

$$\mathrm{H}(M) := \{\, w \in \Sigma^* \mid M \text{ halts given the input } w \,\}.$$

The Halting Problem

The language of the halting problem

Let $\Sigma = \{0, 1\}$. The original Turing machine accepted by halting, no by final state.

$$\mathrm{H}(M) := \{\, w \in \Sigma^* \mid M \text{ halts given the input } w \,\}.$$

We define the language of the halting problem by

$$\mathrm{L}_{\mathrm{hp}} := \left\{\, \overrightarrow{(M, w)} \in \Sigma^* \;\middle|\; w \in \mathrm{H}(M) \,\right\}.$$

The Halting Problem

The language of the halting problem

Let $\Sigma = \{0,1\}$. The original Turing machine accepted by halting, no by final state.

$$\mathrm{H}(M) := \{\, w \in \Sigma^* \mid M \text{ halts given the input } w \,\}.$$

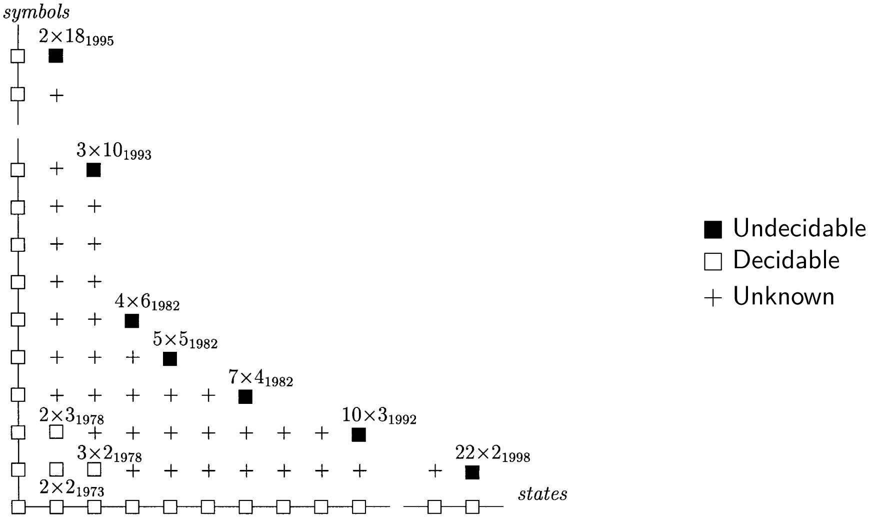
We define the language of the halting problem by

$$\mathrm{L}_{\mathrm{hp}} := \left\{\, \overrightarrow{(M,w)} \in \Sigma^* \;\middle|\; w \in \mathrm{H}(M) \,\right\}.$$

Exercise 9.2.1

Show that L_{hp} is recursively enumerable but not recursive.

The Halting Problem: State of Art†



†Figure from (Margenstern 2000).

The Halting Problem

Remark

Although Turing neither introduced nor named the problem, he *"provided all the core ideas leading to a proof of the undecidability of the halting problem"* (Hamkins and Nenu 2026, p. 15).

The Halting Problem

Remark

Although Turing neither introduced nor named the problem, he *"provided all the core ideas leading to a proof of the undecidability of the halting problem"* (Hamkins and Nenu 2026, p. 15).[†]

[†] The halting problem was introduced by (Kleene 1974, p. 382) and it was named by Davis (1958, p. 70). See (Lucas 2021; Hamkins and Nenu 2026).

The Halting Problem

Practical approach

"In contrast to popular belief, proving termination is not always impossible."
(Cook, Podelski and Rybalchenko 2011, p. 1)

References

Claudio Baiocchi (2001). Three Small Universal Turing Machines. In: Machines, Computations, and Universality (MCU 2001). Ed. by Maurice Margenstern and Yurii Rogozhin. Vol. 2055. Lecture Notes in Computer Science. Springer, pp. 1–10. DOI: 10.1007/3-540-45132-3_1 (cit. on p. 20).

Byron Cook, Andreas Podelski and Andrey Rybalchenko (2011). Proving Program Termination. Communications of the ACM 54.5, pp. 88–98. DOI: 10.1145/1941487.1941509 (cit. on p. 36).

Martin Davis (1958). Computability and Unsolvability. McGraw-Hill (cit. on p. 35).

Joel David Hamkins and Theodor Nenu (2026). Did Turing Prove the Undecidability of the Halting Problem? Journal of Logic and Computation 36.1, exaf075. DOI: 10.1093/logcom/exaf075 (cit. on pp. 34, 35).

Gabor T. Herman (1968). The Uniform Halting Problem for Generalized One State Turing Machines. In: 9th Annual Symposium on Switching and Automata Theory (SWAT 1968). IEEE, pp. 368–372. DOI: 10.1109/SWAT.1968.36 (cit. on p. 22).

John E. Hopcroft, Rajeev Motwani and Jefferey D. Ullman [1979] (2007). Introduction to Automata Theory, Languages, and Computation. 3rd ed. Pearson Education (cit. on pp. 2, 9).

Stephen Cole Kleene [1952] (1974). Introduction to Metamathematics. Seventh reprint. Bibliotheca Mathematica: A Series of Monographs on Pure and Applied Mathematics. Volume 1. North-Holland (cit. on p. 35).

Salvador Lucas (2021). The Origins of the Halting Problem. Journal of Logical and Algebraic Methods in Programming 121, p. 100687. DOI: 10.1016/j.jlamp.2021.100687 (cit. on p. 35).

References

Maurice Margenstern (2000). Frontier Between Decidability and Undecidability: A Survey. Theoretical Computer Science 231.2, pp. 217–251. DOI: 10.1016/S0304-3975(99)00102-4 (cit. on p. 33).

Turlough Neary and Damien Woods (2007). Four Small Turing Machines. In: Machines, Computations, and Universality (MCU 2007). Ed. by Jérôme Durand-Lose and Maurice Margenstern. Vol. 4664. Lecture Notes in Computer Science. Springer, pp. 242–254. DOI: 10.1007/978-3-540-74593-8_21 (cit. on p. 20).

— (2009). Four Small Turing Machines. Fundamenta Informaticae 91.1, pp. 123–144. DOI: 10.3233/FI-2009-0036 (cit. on pp. 20, 21).

— (2012). The Complexity of Small Universal Turing Machines: A Survey. In: Theory and Practice of Computer Science (SOFSEM 2012). Ed. by Mária Bieliková, Gerhard Friedrich, Georg Gottlob, Stefan Katzenbeisser and György Turán. Vol. 7147. Lecture Notes in Computer Science. Springer, pp. 385–405. DOI: 10.1007/978-3-642-27660-6_32 (cit. on pp. 20, 22).

Roger Penrose (1991). The Emperor's New Mind. Penguin Books (cit. on p. 15).

Paul Rendell (2011). A Universal Turing Machine in Conway's Game of Life. In: Proceedings of the 2011 International Conference on High Performance Computing & Simulation (HPCS 2011). Ed. by Waleed W. Smari. IEEE, pp. 764–772. DOI: 10.1109/HPCSim.2011.5999906 (cit. on pp. 25–29).

References

Yurii Rogozhin (1996). Small Universal Turing Machines. Theoretical Computer Science 168, pp. 215–240. DOI: 10.1016/S0304-3975(96)00077-1 (cit. on pp. 20–22).

Claude E. Shannon (1956). A Universal Turing Machine with Two Internal States. In: ed. by C. E. Shannon and J. McCarthy. Vol. 34. Annals of Mathematics Studies. Princenton University Press, pp. 157–165 (cit. on pp. 18, 19).

Andrés Sicard (1997). Máquina Universal de Turing: Algunas Indicaciones para su Construcción. Revista Universidad EAFIT 33.108, pp. 61–106 (cit. on p. 17).

Andrés Sicard and B. Jack Copeland (2004). Appendix: Subroutines and M-functions. In: vol. 317. 1–3, pp. 54–57. DOI: 10.1016/j.tcs.2003.12.014 (cit. on p. 17).

Andrés Sicard Ramírez (1998). Máquinas de Turing Dinámicas: Historia y Desarrollo de una Idea. MA thesis. Departamento de Informática y Sistemas. Universidad EAFIT (cit. on p. 17).